

Professores:

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

Navegação



Capacidades Técnicas Abordadas

3. Interpretar os requisitos do sistema, tendo em vista a elaboração dos componentes em ambiente mobile
4. Definir os elementos de entrada, processamento e saída para a codificação das funcionalidades mobile
5. Projetar interfaces para dispositivos móveis
6. Implementar o código respeitando as características da linguagem na plataforma mobile

O problema

Atualmente para testar nossos exemplos temos que ficar trocando manualmente no **App.js**, em sistemas de produção isso não acontece.

Por isso temos que automatizar esse processo trabalhando com a navegação

```
export default function App() {  
  return (  
    <View>  
      <TelaMoeda /> { /* troca manual */}  
    </View>  
  );  
}
```

Ferramenta

React Navigation



Essa é a biblioteca mais famosa e utilizada no mercado para navegação entre páginas no **React Native**.

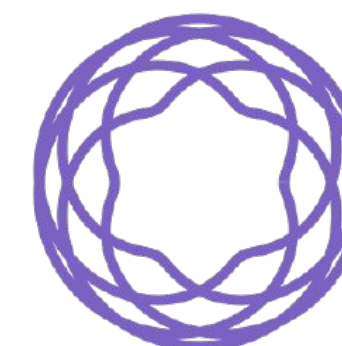
Ela fornece:

- Estrutura para definir as telas como rotas;
- Histórico de navegação (botão de voltar);
- Diferentes tipos de navegações – pilha, abas, gaveta etc.

Por padrão ela não vem instalada, portanto devemos utilizar os comandos:

npm install @react-navigation/native

npx expo install react-native-screens react-native-safe-area-context



NavigationContainer

Para configuração inicial, o primeiro passo é definir um **NavigationContainer** na raiz do nosso projeto.

É importante saber que **só pode EXISTIR UM NavigationContainer** em todo aplicativo. Ele será o nosso contexto de navegação global.

```
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      {/* navegadores ficam aqui dentro */}
    </NavigationContainer>
  );
}
```


Configuração Inicial

Para simular as nossas telas, vou enviar para vocês 4 componentes no classroom, cada um será colocado em um arquivo.:

- HomeScreen;
- DetalheScreen;
- ConfigScreen;
- PerfilScreen.

Esses componentes vão nos ajudar a entender melhor a navegação, vamos utiliza-los para realizar esse processo.

Stack Navigation

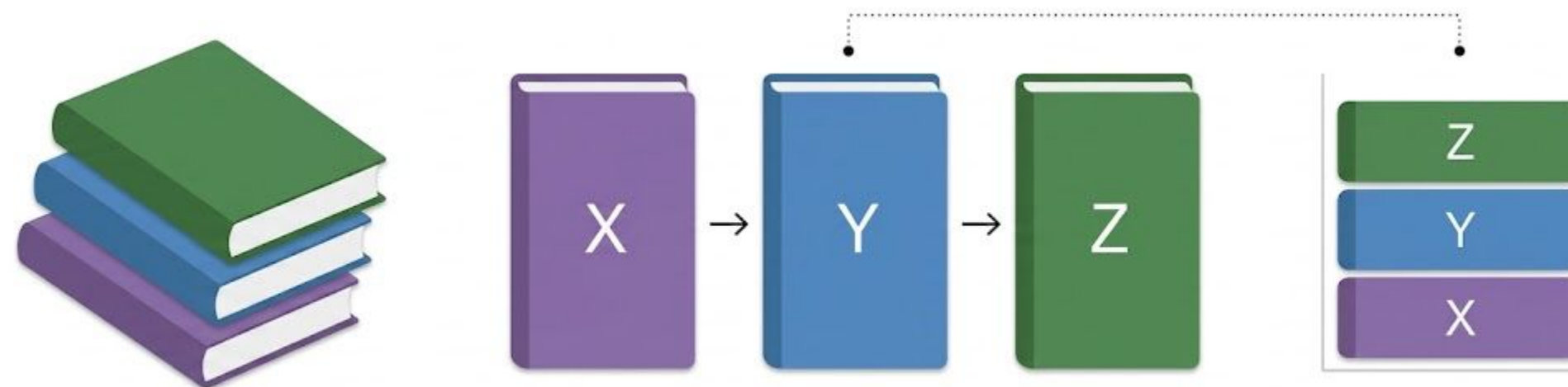


StackNavigation

Também conhecida como navegação em pilha, permite que as telas sejam empilhadas.

- Cada tela entra no topo da pilha;
- Voltar remove a tela do topo;
- O usuário sempre vê a tela que está no topo.

Precisamos instalar esse módulo rodando: **npm install @react-navigation/native-stack**



Como funciona – StackNavigation

Vamos criar um arquivo a parte nomeado como `stack_navigation.js`, nesse arquivo iremos declarar nossas rotas. **Exemplo:**

```
const Stack = createNativeStackNavigator();  
export default function StackNavigator() {  
  return (  
    <Stack.Navigator initialRouteName="Home">  
      <Stack.Screen name="Home" component={HomeScreen} />  
      <Stack.Screen name="Detalhe" component={DetalheScreen} />  
    </Stack.Navigator>  
  );  
}
```

Explicando – StackNavigation

- **Stack.navigator** – gerenciador da pilha;
- **Stack.screen** – registro das telas;
- **name** – nome da rota (utilizado para navegar);
- **component** – o componente que será renderizado.

```
const Stack = createNativeStackNavigator();  
export default function StackNavigator() {  
  return (  
    <Stack.Navigator initialRouteName="Home">  
      <Stack.Screen name="Home" component={HomeScreen} />  
      <Stack.Screen name="Detalhe" component={DetalheScreen} />  
    </Stack.Navigator>  
  );  
}
```

Agora devemos importar o componente que contém a nossa pilha no **App.js**

```
export default function App() {  
  return (  
    <NavigationContainer>  
      <StackNavigator />  
    </NavigationContainer>  
  );  
}
```

Como trocar de tela?

Depois de configurarmos nossa estrutura, podemos passar como uma **prop** em **TODO** componente registrado o **navigation**.

Dessa forma o React Native nos permite utilizar duas funções de navegação:

- `navigation.navigate()` – vai para outra tela
- `navigation.goBack()` – volta para tela anterior

Exemplo – navigate()

```
export default function TelaInicio({ navigation }) {  
  return (  
    <TouchableOpacity onPress={() => navigation.navigate('Detalhe')}>  
      <Text>Ver detalhes</Text>  
    </TouchableOpacity>  
  );  
}
```

Exemplo – goBack()

```
{/* navigation.goBack() volta para a tela anterior na pilha  
    (equivalente ao botao de voltar do cabecalho) */}  
<Button title="Voltar" onPress={() => navigation.goBack()} />  
</View>
```

Passagem de Parâmetros

Em muitas situações, precisaremos passar informações de uma tela para outra, como por exemplo na edição de um produto, onde precisamos passar qual o **ID** dele para realizar a edição.

No React Navigation podemos definir os parâmetros através do **route.params**.

Vamos realizar a passagem em dois passos principais, mexendo no **componente** e na **rota**.

No componente

No componente precisamos **configurar** quais os parâmetros que serão recebidos através da prop **route**.

```
export default function DetalheScreen({ navigation, route }) {  
  const { titulo, descricao } = route.params ?? {};  
  return (  
    <View style={styles.container}>  
      <Text style={styles.titulo}>{titulo ?? "Detalhe"}</Text>  
      <Text style={styles.titulo}>{descricao ?? "Screen"}</Text>  
      <Button title="Voltar" onPress={() => navigation.goBack()} />  
    </View>  
  );  
}
```

Na rota

Na rota precisamos passar as informações **{ titulo, descricao }** que definimos no componente.

```
<Button
  title="Ir para tela de Detalhes"
  onPress={() =>
    navigation.navigate("Detalhe", {
      titulo: "Hello",
      descricao: "World",
    })
  }
/>
```

Bottom Tab Navigator

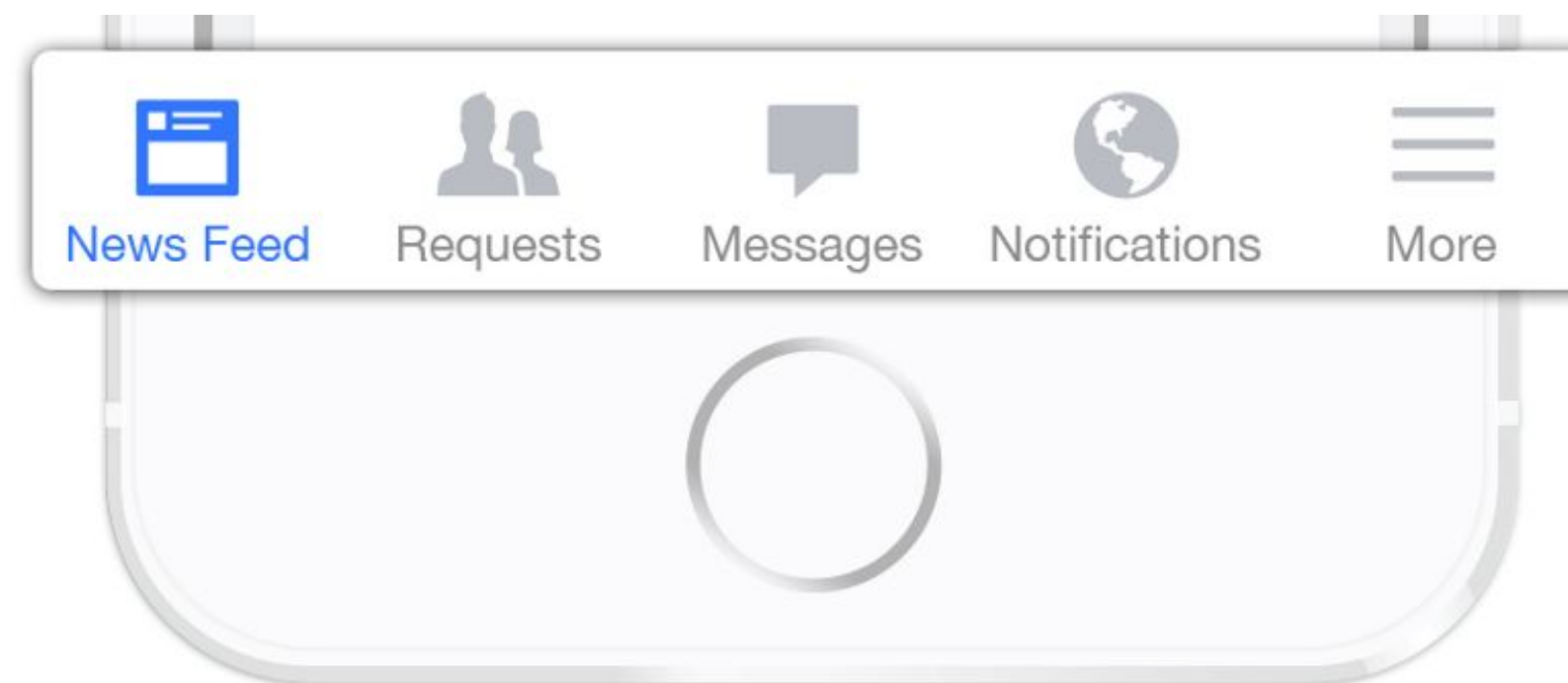


Bottom Tab Navigator

Este é um clássico modo de navegação, onde o usuário possui uma barra na parte inferior do celular com diversas opções.

Utilizado por grandes aplicativos usados diariamente como: Instagram, TikTok, WhatsApp, Spotify etc...

comando: **`npm install @react-navigation/bottom-tabs`**



Bottom Tab Navigator

É muito importante entender como funciona o carregamento da página quando usamos o Bottom Tab.

Nesse tipo de navegação **TODAS AS TELAS SÃO CARREGADAS ASSIM QUE O APP É RENDERIZADO.**

Diferente do Stack Navigation, onde as telas são carregadas conforme o usuário interage com as rotas.

Obs: Nesse modo de navegação os parâmetros não são indicados, pois eles sempre virão como **undefined**.

Implementação

Para implementação vamos criar um novo arquivo: **bottom_tab_navigation.js**

A implementação vai seguir o mesmo padrão, precisamos apenas trocar os nomes e renovar os imports

```
const Tab = createBottomTabNavigator();  
export default function BottomTabNavigator() {  
  return (  
    <Tab.Navigator initialRouteName="Home">  
      <Tab.Screen name="Home" component={HomeScreen} />  
      <Tab.Screen name="Perfil" component={PerfilScreen} />  
    </Tab.Navigator>  
  );  
}
```


Drawer Navigator

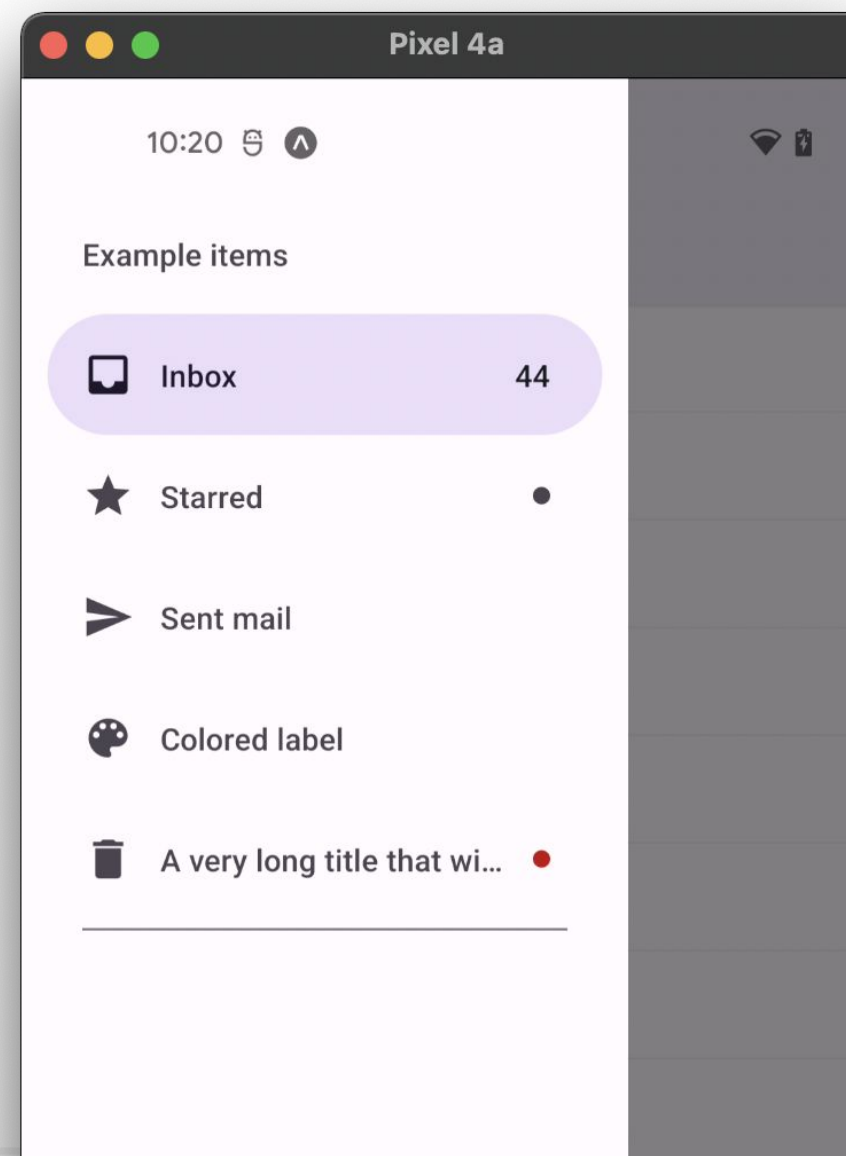


React Navigation

Drawer Navigator



Também é conhecido como menu lateral, muito utilizado em sites e aplicativos desktop como por exemplo: Youtube, Google Maps, Discord, Spotify



Instalação

Esse é o mais complicado de instalar pois possuí diversas dependências.

Primeiro temos que instalar o módulo utilizando o comandos:

- `npm install @react-navigation/drawer`
- `npx expo install react-native-gesture-handler react-native-reanimated react-native-worklets`
- `npm install babel-preset-expo`

Agora na raiz do projeto criar um arquivo: **babel.config.js** contendo:

```
module.exports = {  
  ...  
  presets: ["babel-preset-expo"],  
  plugins: ["react-native-reanimated/plugin"],  
};  
|
```

Implementação

Para implementar vamos criar um arquivo chamado: **drawer_navigation.js** e ajustar os imports

```
const Drawer = createDrawerNavigator();
export default function DrawerNavigator() {
  return (
    <Drawer.Navigator initialRouteName="Home">
      <Drawer.Screen name="Home" component={HomeScreen} />
      <Drawer.Screen name="Perfil" component={PerfilScreen} />
      <Drawer.Screen name="Config" component={ConfigScreen} />
    </Drawer.Navigator>
  );
}
```

Quando usar cada um?

- **Stack:** Navegação é linear, tipo um fluxo de telas que avança e volta (login → cadastro → confirmação, ou lista → detalhes → edição);
- **Bottom Bar:** Quando a app tem 2 a 5 seções principais de mesmo nível de importância que o usuário alterna com frequência (Home, Busca, Perfil, Carrinho);
- **Drawer Navigator:** Quando há muitas seções (6+) ou seções secundárias que não precisam estar sempre visíveis (Configurações, Sobre, Ajuda, Política de Privacidade).

Integração



Mesclando Métodos

Algo muito comum nos aplicativos é mesclar esses métodos de navegação.

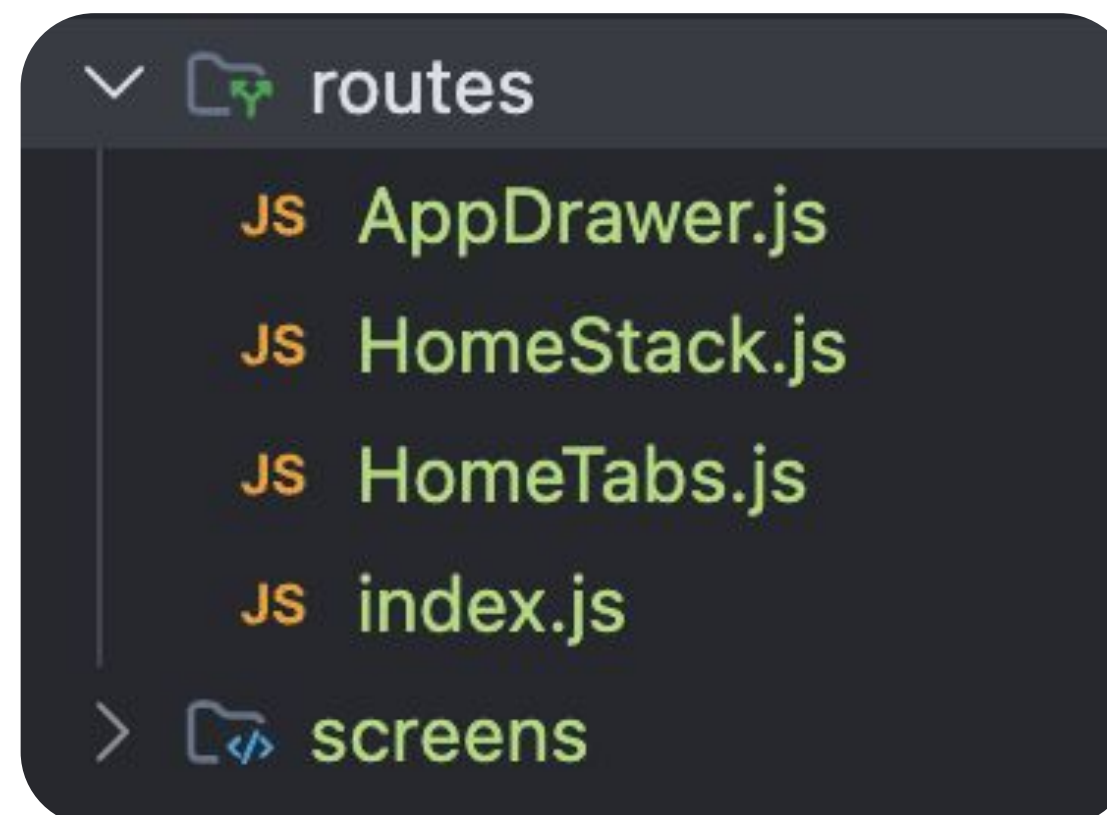
Para fazer isso vou demonstrar novo arquivo chamado **app_routes.js** para centralizar as nossas rotas em um único lugar.

Porém existe um problema já conhecido nessa abordagem.

Estrutura comum para rotas

Como de costume, temos uma estrutura padrão, que geralmente é seguida no mercado.

Essa estrutura tem o intuito de separar as responsabilidades, deixando o **app** chamando apenas um arquivo central para as **rotas**.



Exercício



Exercício: Aplicando navegação

Você recebeu um projeto React Native com 4 telas genéricas.

Sua missão é:

- Implementar navegação entre as telas com React Navigation;
- Implementar hooks (useState e useEffect) nas telas indicadas;
- Escolher um tema e estilizar o app com identidade visual propria;

Passo a Passo

- Instalar as dependências com npm install;
- Implementar pelo menos 2 tipos de navegação (Stack, Tab ou Drawer);
- Adicionar useState e useEffect na HomeScreen e DetalheScreen;
- Substituir dados genéricos pelos dados do seu tema;
- Extrair pelo menos um componente reutilizável;
- Estilizar todas as telas.

Obs: A parte da estilização poderá ser feita com IA, o arquivo .md no repositório sobre essa parte.

Requisitos Técnicos

- Navegação: pelo menos 2 tipos diferentes;
- HomeScreen: busca com useState + filtragem com useEffect;
- DetalheScreen: botão de salvar/remover com estado;
- Estrutura: src/screens, src/navigation, src/components;
- Passagem de parâmetros da HomeScreen para DetalheScreen funcionando.

Referências

FACEBOOK INC. FlatList — React Native Documentation. Disponível em: <https://reactnative.dev/docs/flatlist>. Acesso em: abr. 2026.

FACEBOOK INC. TextInput — React Native Documentation. Disponível em: <https://reactnative.dev/docs/textinput>. Acesso em: abr. 2026.

FACEBOOK INC. Alert — React Native Documentation. Disponível em: <https://reactnative.dev/docs/alert>. Acesso em: abr. 2026.

EXPO. Expo Documentation. Disponível em: <https://docs.expo.dev>. Acesso em: abr. 2026.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>